

Reviewer Pack — Minimal Order of Quaternionic Kähler Manifolds and Communica...

Entry 78fa74bf94c0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 10:06:10 UTC

Minimal Order of Quaternionic Kähler Manifolds and Communication Complexity Rank Variance

Entry ID: 78fa74bf94c0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 10:06:10 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Quaternionic Kähler Manifolds) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given n -input Boolean function, the minimal order of a quaternionic Kähler manifold that can represent the linear transformation of the input space to an output space is polynomially related to the communication complexity rank variance of the function. Specifically, there exists a constant $c > 0$ such that for all n -input functions f , the following holds: $O(\log(n)) \leq \text{Order}(\text{Manifold}(f)) \leq c * \text{CommRankVar}(f)$.

Rationale (proposer's reasoning):

Quaternionic Kähler manifolds provide a geometric representation of complex structures in higher dimensions. They have been studied in algebraic geometry and are known for their rich mathematical properties. By relating the order of such manifolds to communication complexity, we may uncover new structural insights into the complexity of computation tasks that involve distributing information. This conjecture could potentially bridge the gap between geometry and complexity theory, leading to new algorithms or complexity lower bounds.

Taxonomy category: Quaternionic_Kahler_Manifolds_to_Communication_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 16541bbe968e53a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -input Boolean function, if the computed minimal order of the quaternionic Kähler manifold is within the polynomial bounds $O(\log(n))$ and $\leq c * \text{CommRankVar}(f)$, where $\text{CommRankVar}(f)$ is the communication complexity rank variance and c is a constant greater than 0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic Kähler Manifolds" AND "communication complexity" AND matrix rank - "minimal order" quaternionic Kähler manifolds communication complexity rank variance" - polynomial relationship minimal order quaternionic Kähler manifold communication complexity rank variance

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank_variance(f):
        n = int(math.log2(len(f)))
        rank_var = sum((f[i] - f[j])**2 for i in range(len(f)) for j in range(i+1, len(f))) / (len(f) * (len(f) - 1))
        return rank_var

    def minimal_order_of_manifold(n):
        # Placeholder function to simulate the computation
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, n)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        f = generate_boolean_function(n)
        rank_var = communication_complexity_rank_variance(f)
        order = minimal_order_of_manifold(n)
```

```

    if order < n or order > 10 * rank_var: # Placeholder bounds, replace with actual logic
        return {
            "metric_name": "Order(Manifold(f))",
            "metric_value": order,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "bounds_violation"
        }

    total_metric_value += order
    instances_tested += 1
    n_max = max(n_max, n)

mean_metric_value = total_metric_value / instances_tested
return {
    "metric_name": "Order(Manifold(f))",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='bounds_violation' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

: 3, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 2, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'}
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 4, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'}
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 3, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'}
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 2, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'}
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 4, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'bounds_violation'}

```

```

TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 1, 'instances_tested': 0, 'n_max': 0, 'c
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 3, 'instances_tested': 0, 'n_max': 0, 'c
TRIAL: {'metric_name': 'Order(Manifold(f))', 'metric_value': 5, 'instances_tested': 0, 'n_max': 0, 'c

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e2e784.py", line 85, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e2e784.py", line 85, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that no results were obtained to evaluate the conjecture. | next: Investigate and fix the crash in the test code to obtain valid results for further evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13857
2	preregistration	ollama_remote	glm4:latest	0	0	9530
3	novelty	ollama_remote	glm4:latest	0	0	9377
4	novelty	ollama_remote	glm4:latest	0	0	12425
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16561
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7094
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8851
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8661
9	judge	ollama_remote	glm4:latest	0	0	16405

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102760 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/78fa74bf94c0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/78fa74bf94c0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/78fa74bf94c0.tar.gz` (if generated)